

THỰC HÀNH

HƯỚNG DẪN SỬ DỤNG GITHUB CHO NGƯỜI MỚI BẮT ĐẦU

Quản lý mã nguồn và project bằng Git, GitHub, Terminal và Visual Studio Code

Sau bài thực hành, người học có thể tạo repository, clone project, ghi nhận thay đổi bằng commit, đồng bộ code với GitHub và xử lý các lỗi phổ biến.

Phạm vi: Tài liệu ưu tiên môi trường macOS và Terminal zsh. Các lệnh Git cốt lõi cũng áp dụng tương tự trên Windows và Linux.

Ví dụ xuyên suốt: github.com/khoatrantm/java-oop-practice

1. Mục tiêu và kết quả cần đạt

Tài liệu này được xây dựng như một bài thực hành có thể tự học hoặc sử dụng trực tiếp trong lớp.

- Phân biệt được Git và GitHub.
- Hiểu repository, commit, branch, clone, push, pull và Pull Request.
- Tạo repository mới và tải project từ GitHub về máy.
- Xác thực an toàn bằng Personal Access Token khi cần thiết.
- Mở project bằng Visual Studio Code và khắc phục lỗi lệnh code không hoạt động.
- Thực hiện quy trình cập nhật code hằng ngày và làm việc theo branch.
- Nhận biết và xử lý các lỗi Git/GitHub thường gặp.

2. GitHub dùng để làm gì?

Git là công cụ quản lý phiên bản trên máy tính; GitHub là nền tảng trực tuyến dùng để lưu trữ repository Git và hỗ trợ cộng tác.

Chức năng	Ý nghĩa thực tế
Lưu trữ mã nguồn	Đưa code và tài liệu project lên repository Public hoặc Private.
Quản lý phiên bản	Lưu lịch sử thay đổi bằng commit; biết ai sửa, sửa gì và khi nào.
Làm việc nhóm	Phát triển trên branch, review bằng Pull Request và hợp nhất vào main.
Quản lý công việc	Theo dõi lỗi và nhiệm vụ bằng Issues, Labels, Milestones và Projects.
Tự động hóa	Build, test hoặc triển khai bằng GitHub Actions.
Công bố sản phẩm	Phát hành phiên bản bằng Tags, Releases hoặc GitHub Pages.

3. Thuật ngữ cơ bản

Thuật ngữ	Giải thích ngắn
Repository (repo)	Thư mục project được Git quản lý.
Local repository	Bản project nằm trên máy tính.
Remote repository	Bản project trực tuyến, thường nằm trên GitHub.
Commit	Một mốc lưu các thay đổi có nội dung mô tả.

Thuật ngữ	Giải thích ngắn
Branch	Nhánh phát triển độc lập; nhánh chính thường là main.
Clone	Tải toàn bộ repository và lịch sử về máy.
Push	Đưa commit từ máy lên GitHub.
Pull	Lấy thay đổi mới từ GitHub về máy.
Pull Request	Đề nghị review và hợp nhất một branch.
Conflict	Xung đột khi hai thay đổi tác động cùng một vùng nội dung.

4. Chuẩn bị môi trường

BƯỚC 1 Tạo tài khoản GitHub

1. Truy cập <https://github.com> và đăng ký tài khoản.
2. Xác minh địa chỉ email.
3. Bật xác thực hai bước (2FA) để bảo vệ tài khoản.

BƯỚC 2 Cài đặt công cụ

- Git: <https://git-scm.com/downloads>
- Visual Studio Code: <https://code.visualstudio.com>
- GitHub Desktop (tùy chọn): <https://desktop.github.com>

BƯỚC 3 Kiểm tra Git

```
git --version
```

BƯỚC 4 Khai báo tác giả commit

```
git config --global user.name "Tran Thi Minh Khoa"
git config --global user.email "email@example.com"
git config --global init.defaultBranch main
git config --global --list
```

Phân biệt: user.name là tên tác giả hiển thị trong commit; user.email là email gắn với commit. Hai giá trị này không phải URL dùng để clone repository.

5. Bài thực hành 1 - Tạo và clone repository

BƯỚC 1 Tạo repository trên GitHub

1. Đăng nhập GitHub và chọn dấu + ở góc trên bên phải.
2. Chọn New repository.
3. Nhập tên repository, ví dụ java-oop-practice.
4. Chọn Public hoặc Private.
5. Có thể chọn Add a README file và mẫu .gitignore phù hợp.
6. Nhấn Create repository.

BƯỚC 2 Sao chép URL chính xác

1. Mở repository vừa tạo.
2. Nhấn Code.
3. Chọn HTTPS và nhấn biểu tượng Copy.

Username là gì?: Trong URL `https://github.com/khoatrantm/java-oop-practice.git`, username là `khoatrantm`. Đây là tên tài khoản GitHub, không phải họ tên hiển thị và không phải địa chỉ email.

BƯỚC 3 Chọn thư mục muốn lưu project

```
mkdir -p ~/Documents/GitHub
cd ~/Documents/GitHub
pwd
```

BƯỚC 4 Thực hiện clone

```
git clone https://github.com/khoatrantm/java-oop-practice.git
```

BƯỚC 5 Đi vào project và kiểm tra

```
cd java-oop-practice
ls
git status
```

6. Project clone về được lưu ở đâu?

Git tạo một thư mục mới mang tên repository bên trong thư mục hiện tại của Terminal.

Lệnh / ký hiệu	Ý nghĩa
<code>pwd</code>	Hiển thị đường dẫn thư mục hiện tại.
<code>ls</code>	Liệt kê file và thư mục tại vị trí hiện tại.
<code>~</code>	Thư mục cá nhân trên macOS, ví dụ <code>/Users/kh</code> .

Lệnh / ký hiệu	Ý nghĩa
cd tên-thư-mục	Di chuyển Terminal vào thư mục chỉ định.
open .	Mở thư mục hiện tại bằng Finder.

```
# Nếu clone khi Terminal đang ở ~
~/java-oop-practice

# Nếu đã chuyển vào Documents/GitHub trước khi clone
~/Documents/GitHub/java-oop-practice
```

7. Xác thực bằng Personal Access Token (PAT)

PAT là chuỗi bí mật do GitHub tạo ra để thay cho mật khẩu khi Git xác thực qua HTTPS. GitHub không chấp nhận mật khẩu tài khoản thông thường cho thao tác Git qua HTTPS.

Repository Public: Clone một repository Public thông thường không cần username hoặc token. Nếu được hỏi thông tin xác thực, repository có thể đang ở chế độ Private hoặc tài khoản chưa có quyền truy cập.

7.1. Khi Terminal hỏi username và password

```
Username for 'https://github.com': khoatrantm
Password for 'https://khoatrantm@github.com': <dán PAT tại đây>
```

- Username: nhập tên tài khoản GitHub, ví dụ khoatrantm.
- Password: dán Personal Access Token, không nhập mật khẩu GitHub.
- Khi dán token, Terminal không hiển thị ký tự hoặc dấu sao; đây là hành vi bình thường.

7.2. Cách tạo Fine-grained PAT

1. Đăng nhập GitHub và mở Settings.
2. Chọn Developer settings ở cuối menu bên trái.
3. Chọn Personal access tokens > Fine-grained tokens.
4. Nhấn Generate new token.
5. Đặt Token name, ví dụ MacBook Git Access, và chọn ngày hết hạn.
6. Tại Resource owner, chọn tài khoản của mình.
7. Tại Repository access, chọn Only select repositories và chọn java-oop-practice.
8. Tại Repository permissions > Contents, chọn Read and write nếu cần clone, pull và push.
9. Nhấn Generate token, sao chép token và lưu ở nơi an toàn.

Bảo mật: Không gửi token, không chụp ảnh token và không lưu token trong source code, README hoặc file .env được commit. Nếu token bị lộ, phải thu hồi token ngay trong GitHub Settings.

8. Mở project bằng Visual Studio Code

BƯỚC 1 Đi vào thư mục project

```
cd ~/Documents/GitHub/java-oop-practice
```

BƯỚC 2 Mở project

```
code .
```

Dấu chấm (.) có nghĩa là thư mục hiện tại. Vì vậy code . yêu cầu VS Code mở toàn bộ thư mục mà Terminal đang đứng.

8.1. Nếu lệnh code không hoạt động

1. Mở Visual Studio Code từ Applications hoặc Launchpad.
2. Nhấn Command + Shift + P để mở Command Palette.
3. Nhập Shell Command.
4. Chọn Shell Command: Install 'code' command in PATH.
5. Đóng Terminal, mở Terminal mới và kiểm tra bằng code --version.

```
code --version
cd ~/Documents/GitHub/java-oop-practice
code .
```

Cách mở ngay: Nếu chưa cấu hình lệnh code, có thể dùng: open -a "Visual Studio Code" .

```
open -a "Visual Studio Code" .
```

9. Quy trình cập nhật code hằng ngày

Mỗi lần làm việc, nên lấy phiên bản mới nhất trước, kiểm tra thay đổi, tạo commit rõ nghĩa rồi mới push lên GitHub.

```
cd ~/đường_dẫn_đến_thư_mục_dự_án
git pull --rebase origin main //cập nhật thay đổi mới nhất từ GitHub

# Chỉnh sửa source code trong VS Code

git status //kiểm tra những file đã thay đổi
git diff //xem nội dung thay đổi chi tiết
git add . //đưa tất cả file thay đổi vào, hoặc chỉ một file: git add
src/SanPham.java
git commit -m "Add student validation" //tạo commit kèm mô tả
```

```
git pull --rebase origin main //đồng bộ lại lần nữa trước khi push
git push origin main
```

Lệnh	Tác dụng
git status	Xem branch hiện tại và các file đã thay đổi.
git diff	Xem nội dung thay đổi chưa đưa vào staging.
git add File.java	Chọn một file để chuẩn bị commit.
git add .	Chọn tất cả thay đổi trong project.
git commit -m "..."	Tạo một mốc phiên bản kèm mô tả.
git push	Đưa commit lên GitHub.
git pull	Lấy và hợp nhất thay đổi từ GitHub về máy.

Commit tốt: Mỗi commit nên chứa một thay đổi hoàn chỉnh và có mô tả cụ thể, ví dụ Add product validation hoặc Fix total calculation. Tránh các nội dung mơ hồ như update, abc hoặc fix.

10. Làm việc bằng branch và Pull Request

Branch giúp phát triển chức năng mới mà không làm ảnh hưởng trực tiếp đến nhánh main.

```
git switch main
git pull origin main
git switch -c feature/student-management

# Chỉnh sửa code

git add .
git commit -m "Add student management feature"
git push -u origin feature/student-management
```

1. Mở repository trên GitHub.
2. Chọn Compare & pull request.
3. Kiểm tra base là main và compare là feature/student-management.
4. Viết tiêu đề, mô tả thay đổi và tạo Pull Request.
5. Review code, xử lý nhận xét rồi chọn Merge pull request.
6. Sau khi merge, cập nhật nhánh main trên máy và xóa branch cũ.

```
git switch main
git pull origin main
git branch -d feature/student-management
```

11. Đưa một project có sẵn lên GitHub

Trường hợp project đã tồn tại trên máy nhưng chưa được Git quản lý, hãy tạo một repository trống trên GitHub. Không chọn README hoặc .gitignore khi tạo remote để tránh hai lịch sử khởi tạo độc lập.

```
cd /duong-dan-den-project
git init
git add .
git commit -m "Initial commit"
git branch -M main
git remote add origin https://github.com/username/ten-project.git
git push -u origin main
```

Thay giá trị mẫu: Phải thay username và ten-project bằng tên tài khoản và tên repository thực tế. Cách an toàn nhất là sao chép URL từ Code > HTTPS trên GitHub.

12. README và .gitignore

12.1. README.md

README là trang giới thiệu project được GitHub hiển thị ngay tại trang repository. Nội dung nên có tên project, chức năng, công nghệ, hướng dẫn cài đặt và cách chạy.

```
# Student Management System

Ứng dụng quản lý sinh viên bằng Java.

## Chức năng
- Thêm, sửa, xóa sinh viên
- Tìm kiếm sinh viên

## Công nghệ
- Java 21
- Maven

## Cách chạy
1. Clone repository.
2. Mở project bằng VS Code.
3. Chạy lớp Main.
```

12.2. .gitignore

.gitignore quy định những file Git không theo dõi. Không đưa mật khẩu, token, API key hoặc dữ liệu nhạy cảm lên GitHub.

```
# Bí mật
.env
application-secret.properties

# Java
*.class
target/
build/

# Node.js
node_modules/
dist/

# Python
__pycache__/_
.venv/

# macOS
.DS_Store
```

13. Quản lý công việc trên GitHub

Công cụ	Cách sử dụng
Issues	Ghi nhận lỗi, yêu cầu chức năng hoặc nhiệm vụ.
Labels	Phân loại issue: bug, feature, documentation, priority.
Assignees	Giao issue cho thành viên phụ trách.
Milestones	Nhóm issue theo phiên bản hoặc giai đoạn.
Projects	Quản lý trạng thái Backlog, To do, In progress, Review, Done.

Liên kết công việc: Trong mô tả Pull Request, ghi Closes #12 để GitHub tự đóng Issue số 12 sau khi Pull Request được merge.

14. Xử lý lỗi thường gặp

Thông báo	Hướng xử lý
zsh: no such file or directory: https://...	Đã nhập riêng URL. Chạy git clone URL thay vì chạy URL.
Repository not found	Kiểm tra chính tả URL, repository đã tồn tại, chế độ Private và quyền truy cập.
destination path ... already exists	Thư mục cùng tên đã tồn tại. Đi vào thư mục cũ hoặc clone với tên thư mục khác.
fatal: not a git repository	Terminal đang đứng sai thư mục hoặc thư mục chưa được git init.
command not found: code	Cài Shell Command: Install 'code' command in PATH hoặc dùng open -a "Visual Studio Code" .
remote origin already exists	Kiểm tra git remote -v rồi cập nhật URL bằng git remote set-url origin URL.
Push bị rejected	Lấy thay đổi mới bằng git pull --rebase origin main, xử lý conflict rồi push lại.

14.1. Các lệnh chẩn đoán

```
pwd
ls
git status
git branch
git remote -v
git log --oneline --decorate -10
```

14.2. Xử lý merge conflict

Khi cùng một vùng code bị sửa khác nhau, Git đánh dấu xung đột trong file. Chọn nội dung đúng, xóa các dấu đánh dấu, lưu file rồi commit lại.

```
<<<<<<< HEAD
Nội dung trên branch hiện tại
=====
Nội dung từ branch được hợp nhất
>>>>>> feature/login

git add .
git commit -m "Resolve merge conflict"
git push
```

15. Hoàn tác thay đổi an toàn

Nhu cầu	Lệnh
Bỏ thay đổi chưa commit của một file	<code>git restore Main.java</code>
Bỏ file khỏi staging nhưng giữ nội dung sửa	<code>git restore --staged Main.java</code>
Xem lịch sử commit ngắn gọn	<code>git log --oneline</code>
Hoàn tác một commit đã chia sẻ	<code>git revert COMMIT_ID</code>

Thận trọng: Không dùng `git reset --hard` hoặc `git push --force` khi chưa hiểu rõ hậu quả. Hai lệnh này có thể làm mất thay đổi hoặc viết lại lịch sử chung của nhóm.

16. Bài tập thực hành tổng hợp

Thực hiện bài tập sau để hoàn thành toàn bộ chu trình GitHub cơ bản.

1. Tạo repository `github-practice` trên GitHub và thêm `README.md`.
2. Clone repository về `~/Documents/GitHub`.
3. Mở project bằng VS Code.
4. Tạo file `Main.java` và in dòng `Hello GitHub`.
5. Kiểm tra thay đổi bằng `git status` và `git diff`.
6. Tạo commit với thông điệp `Add initial Java program`.
7. Push commit lên GitHub.
8. Tạo branch `feature/introduction`.
9. Bổ sung phần giới thiệu vào `README.md`, commit và push branch.
10. Tạo Pull Request, merge vào main và cập nhật nhánh main trên máy.
11. Tạo một Issue mô tả chức năng sẽ phát triển tiếp theo.

16.1. Sản phẩm cần nộp

- Đường dẫn repository GitHub.
- `README.md` có mô tả project và hướng dẫn chạy.
- Tối thiểu 3 commit có nội dung rõ ràng.
- Ít nhất 1 branch đã được merge bằng Pull Request.
- Ít nhất 1 Issue được tạo.

16.2. Checklist tự đánh giá

Tiêu chí	Đạt
Clone repository đúng thư mục mong muốn	☐
Phân biệt GitHub username và email	☐
Không nhập mật khẩu GitHub thay cho PAT	☐
Mở được project bằng VS Code	☐
Thực hiện được add, commit, push và pull	☐
Tạo và merge Pull Request	☐
Không đưa token hoặc file bí mật lên GitHub	☐

Phụ lục - Bảng lệnh Git nhanh

Lệnh	Tác dụng
git clone URL	Tải repository về máy
git status	Kiểm tra trạng thái
git diff	Xem nội dung thay đổi
git add .	Đưa thay đổi vào staging
git commit -m "message"	Tạo commit
git push	Đẩy commit lên GitHub
git pull	Lấy thay đổi về máy
git switch -c tên-branch	Tạo và chuyển branch
git branch	Liệt kê branch
git remote -v	Kiểm tra remote
git log --oneline	Xem lịch sử commit
git restore file	Bỏ thay đổi chưa commit

Tài liệu tham khảo

- GitHub Docs - Get started: <https://docs.github.com/en/get-started>
- GitHub Docs - Hello World: <https://docs.github.com/en/get-started/using-github/hello-world>
- GitHub Docs - Managing personal access tokens:
<https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/managing-your-personal-access-tokens>
- GitHub Docs - GitHub Flow: <https://docs.github.com/en/get-started/using-github/github-flow>